Documento de Diseno y Justificacion

Sistema de Reservacion de Boletos con Bloqueo Temporal de Asientos

Parcial 2 — Desarrollo e Integracion de Aplicaciones con Bases de Datos NoSQL en la Nube
Modelo NoSQL: Clave-Valor | Servicio: Redis Cloud (managed)
Marcos — Universidad Anahuac Mayab

---

1. Introduccion y Dominio del Problema

El caso de uso elegido es la venta de boletos para eventos en vivo (conciertos, teatro, deportes), donde multiples usuarios pueden intentar reservar el mismo asiento de forma simultanea. El sistema debe resolver tres necesidades funcionales:

• Permitir que un cliente "aparte" un asiento por un tiempo limitado mientras completa su pago.
• Liberar automaticamente el asiento si el pago no se confirma a tiempo, evitando asientos "fantasma" bloqueados para siempre.
• Soportar una lista de espera ordenada cuando el evento esta agotado.

2. Justificacion del Paradigma: Clave-Valor vs. Relacional (SQL)

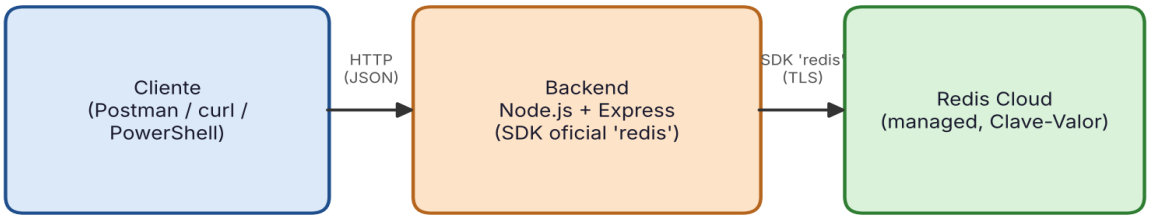
Un SGBD relacional modelaria este problema con tablas *events*, *seats*, *holds* y *bookings* relacionadas por llaves foraneas. Sin embargo, el requisito critico del dominio — "este bloqueo debe expirar solo, despues de N segundos, sin intervencion externa" — no existe de forma nativa en SQL: requeriria un proceso externo (cron/job) que revise periodicamente que bloqueos ya vencieron, con riesgo real de condiciones de carrera si dos clientes intentan confirmar el mismo asiento en la ventana entre revisiones.

El modelo Clave-Valor (Redis) resuelve esto de forma nativa y con mejor rendimiento:

• **Atomicidad del bloqueo:** el comando `SET ... NX EX <segundos>` crea el bloqueo y su expiracion en una sola operacion atomica. Si dos clientes intentan bloquear el mismo asiento al mismo milisegundo, *NX* ("solo si no existe") garantiza que solo uno tenga exito, sin necesidad de un semaforo externo.
• **Expiracion nativa (TTL):** el propio motor de Redis elimina el bloqueo vencido; no se requiere un proceso externo que "limpie" datos caducados.
• **Patron de acceso por llave exacta:** el dominio no necesita joins complejos (el acceso dominante es "dame el asiento X del evento Y"), por lo que la latencia es O(1) en vez del costo de un join relacional bajo alta concurrencia.
• **Throughput:** un lanzamiento de boletos puede generar miles de intentos de reserva por segundo; Redis, al operar en memoria con un modelo de ejecucion sin locks de fila, esta disenado exactamente para ese patron de carga.

3. Arquitectura de la Solucion

El backend centraliza toda la logica de negocio; el cliente nunca accede a Redis directamente. Las credenciales de conexion se inyectan via variables de entorno (`.env`), nunca quedan escritas en el codigo fuente.


Cliente
(Postman / curl /
PowerShell)
HTTP
(JSON)
Backend
Node.js + Express
(SDK oficial 'redis')
SDK 'redis'
(TLS)
Redis Cloud
(managed, Clave-Valor)

4. Diseno del Modelo NoSQL (Clave-Valor)

4.1 Convencion de nombres de llaves (namespacing):

Llave	Tipo Redis	Contenido
event:{eventId}	Hash	name, venue, date, price
event:{eventId}:seats	Set	ids de todos los asientos del evento
seat:{eventId}:{seatId}	Hash	status (available/held/booked), section, row
hold:{eventId}:{seatId}	String + TTL	customerId que sostiene el bloqueo temporal
booking:{bookingId}	Hash	eventId, seatId, customerId, status, createdAt
waitlist:{eventId}	Sorted Set	customerId, score = timestamp de solicitud

4.2 Estrategia de expiracion (TTL):

La llave `hold:{eventId}:{seatId}` se crea con `EX 300` (5 minutos, configurable via la variable de entorno `HOLD_TTL_SECONDS`). Si el cliente no confirma el pago en ese tiempo, Redis elimina la llave automaticamente y el asiento vuelve a quedar disponible para los demas usuarios.

4.3 Politica de persistencia:

Redis Cloud (free tier) usa snapshots periodicos (RDB) por defecto del proveedor, lo cual es suficiente para este caso de uso: los bloqueos (*holds*) son intencionalmente efimeros, mientras que los datos que si deben persistir — eventos, asientos y reservaciones confirmadas — no llevan TTL y quedan guardados de forma permanente.

5. Catalogo de Consultas Clave (SDK 'redis' para Node.js)

```
// Crear un bloqueo atomico con expiracion (TTL) — la consulta especializada central del proyecto
await client.set(`hold:${'{'}eventId{'}'}:${'{'}seatId{'}'}`, customerId, {{ NX: true, EX: 300 }});

// Verificar cuanto tiempo le queda a un bloqueo
const ttl = await client.ttl(`hold:${'{'}eventId{'}'}:${'{'}seatId{'}'}`);

// Confirmar una reservacion (estructura Hash)
await client.hSet(`booking:${'{'}bookingId{'}'}`, {{ eventId, seatId, customerId, status: 'confirmed'
}});

// Agregar un cliente a la lista de espera (Sorted Set, ordenado por tiempo de solicitud)
await client.zAdd(`waitlist:${'{'}eventId{'}'}`, {{ score: Date.now(), value: customerId }});

// Leer la lista de espera en orden de llegada
await client.zRangeWithScores(`waitlist:${'{'}eventId{'}'}`, 0, -1);
```

El codigo fuente completo, el README con la guia de instalacion y el script de poblado inicial (seeding) se encuentran en el repositorio de GitHub del proyecto.